

Claude Code 学習ノート

～ はじめての用語まとめ ～

プログラミングの知識がゼロでも分かるように、
身近な例えを使ってやさしく解説しました。

作成日:2026 年 6 月 11 日

はじめに — この文書の使い方

この文書は、Claude Code や Web 開発の勉強で出てくる用語を、「知識ゼロの人でも納得・理解できる」ことを目指してまとめたものです。

むずかしい言葉は、できるだけ身近なもの(レストラン・お風呂・連絡先アプリ など)にたとえて説明しています。一度で覚えようとせず、分からなくなったら何度でも読み返してください。

◆ 用語は大きく 3 つのグループに分かれます

- ① Web システムを作る「技術」: Spring Boot / REST API / JSON / エンドポイント など
- ② 開発でよく使う「一般用語」: デモ / タスク / バッファ / ロジック / IDE
- ③ 作るときの「流れ・仕組み」: 開発の流れ / 認証(JWT) / エンティティ設計 / 例外ハンドラ

最後に「全体まとめ表」と「全体のつながり図」を載せています。全体像を確認したいときは、そちらから見るのもおすすめです。

第 1 部 — Web システムを作る「技術」

1. Spring Boot(スプリング ブート)

ひとことで言うと:Java で「裏側のシステム」をすばやく作るための、便利な道具セット。

Spring Boot は、Java というプログラミング言語で、Web システムやアプリを作るための「道具箱(フレームワーク)」です。フレームワークとは、よく使う機能があらかじめ用意された「開発の土台・骨組み」のこと。一から全部作らなくても、便利な部品の上に乗って作れます。たとえるなら「だし」や「調味料」が最初から揃っているキッチン。一から塩を作る必要がなく、すぐ料理(開発)に取りかかれます。

◆ うれしいポイント

- 設定がほぼ自動 — 面倒な準備を「いい感じ」に自動でやってくれる
- サーバーが内蔵 — 別途サーバーを用意しなくても、実行するだけで Web システムが立ち上がる
- すぐ動かせる — 少ないコードで、すぐ「動くもの」が作れる

【つながり】 この Spring Boot を使って、次の「REST API」を作るのが定番の組み合わせです。

2. REST API(レスト エーピーアイ)

ひとことで言うと:プログラム同士が「住所」と「動詞」を使ってやり取りする窓口・仕組み。

まず API とは「プログラム同士が会話するための“窓口”」のこと。たとえば天気アプリは、裏側で気象データを持つ別のコンピュータに「東京の天気ちょうだい!」とお願いして表示しています。この“お願いの窓口”が API です。

REST とは、その API の作り方の中で一番人気のお作法のこと。次の 2 つを組み合わせで「誰に・何をしてほしいか」を表現します。

動詞	意味	レストランの例え
GET	取得する(見たい)	メニューを見せて
POST	新しく作る	新しく注文する
PUT	更新する(書き換える)	さっきの注文を変更して
DELETE	削除する	注文をキャンセルして

例: 「/users/123」という住所(ユーザー123 番さん)に対して、GET なら“見せて”、PUT なら“書き換えて”、DELETE なら“消して”という意味になります。

返ってくる答え(データ)は、次の JSON という形式がよく使われます。

【つながり】 「住所(エンドポイント)+ 動詞」で表現します。エンドポイントは後の章で説明します。

3. JSON(ジェイソン)

ひとことで言うと:データ(情報)を書き表すための、世界共通の「書き方の形式」。

JSON は JavaScript Object Notation の略。プログラム同士が情報をやり取りするとき、「みんなこの書き方にしようね」と決められた共通フォーマットです。人間が見ても分かりやすく、プログラムも正確に読めるのが特長です。

「田中太郎さん、28 歳、東京在住」を JSON で書くと、こうなります:

```
{
  "name": "田中太郎",
  "age": 28,
  "city": "東京"
}
```

◆ 基本ルール

- 「キー:バリュー」のペアで書く —— 左が名前(キー)、右が中身(値)
- 全体を {} で囲む
- 複数は , で区切る —— ただし最後の項目の後ろにはカンマを付けない

- 文字は " " で囲む / 数字はそのまま

さらに、複数を並べる [] (リスト/配列) や、{ } の中にさらに { } を入れる **入れ子** もできます。

【つながり】 REST API のお願いの“答え”は、たいていこの JSON 形式で返ってきます。

Claude Code の設定ファイル(settings.json)も JSON 形式です。

4. エンドポイント(Endpoint)

ひとことと言うと:API の窓口一つひとつの「住所(URL)」のこと。

エンドポイント(end=終わり、point=点)は、お願いを送る“宛先”です。REST API 全体が「お店」だとすると、エンドポイントは **一つひとつの注文カウンター(窓口)**にあたります。

たとえば会員管理システムなら、こんなエンドポイントを用意します:

GET	/users	→ 会員一覧を見る
GET	/users/123	→ 123 番の会員を 1 人見る
POST	/users	→ 会員を新しく登録する
DELETE	/users/123	→ 123 番の会員を削除する

ポイントは、**同じ住所 /users でも、付ける動詞(GET / POST…)**が違えば別の意味になること。住所と動詞をセットで考えるのがコツです。

【つながり】 「どんなエンドポイントを用意するか」を先に決めるのが、後で出てくる『API 設計』です。

第2部 — 開発でよく使う「一般用語」

5. デモ(Demo)

ひとことと言うと:実際に動かして見せる「実演」や「お試し版」のこと。

デモは Demonstration(実演)の略。家電量販店で店員さんが掃除機を実際に動かして「こんなに吸いますよ!」と見せてくれる、あれがデモです。説明だけでなく“本当に動くところを見せる”のがポイント。

- 作ったものを動く状態で見せること(「デモを見せる」)
- 機能を限定したお試し版(ゲームの体験版など)
- 「こう動きますよ」という見本のプログラム

【つながり】 Claude Code で作った機能を「実際に動かして確認する」のも、デモの一種です。

6. タスク(Task)

ひとこと言うとは:やるべき作業1つ分。To-Do リストの各項目のようなもの。

「牛乳を買う」「部屋を掃除する」——この一つひとつが1タスクです。ITの世界では、主に2つの意味で使われます。

- **人の作業:**「ログイン画面を作る」など、開発の仕事を小さく分けた1つ分
- **コンピュータの処理:**「画像を保存する」など、コンピュータがこなす1つの処理

Windowsの「タスクマネージャー」は、コンピュータが今こなしている処理(タスク)を管理する画面です。

【つながり】 大きなお願いを、いくつかの小さなタスクに分けて、ひとつずつ進めるのが基本の進め方です。

7. バッファ(Buffer)

ひとこと言うとは:一時的にデータをためておく場所、または「余裕・ゆとり」。

お風呂の浴槽を想像してください。蛇口の水の勢いがバラバラでも、いったん浴槽にためておけばいつでも安定して使えます。この“**いったんためておく場所**”がバッファです。

動画の「読み込み中…」は、データを少し先までためておいて、ネットが遅くなっても途切れないようにする仕組み。これを「バッファリング」と呼びます。

もう一つの意味は「余裕」。「**スケジュールにバッファを持たせる**」＝予定がずれても大丈夫なように、少し予備の時間を入れておく、という意味です。

【**つながり**】 共通イメージは「ムラや変化を吸収するクッション」。文脈で意味を読み分けましょう。

8. ロジック(Logic)

ひとことで言うと:プログラムの「**考え方・処理の中身・手順**」のこと。

ロジックは日本語で「論理」。プログラムが「**こういう時はこうする**」という判断や計算の**中身・段取り**を指します。料理でいう“レシピの手順”のようなものです。

たとえば「会員登録」の処理なら、こんなロジック(手順)になります:

- ① 入力された内容が正しいか確認する
- ② すでに同じ会員がいないか調べる
- ③ 問題なければデータベースに保存する
- ④ 「登録できました」と結果を返す

「見た目(画面)」ではなく、“**裏側で何をどう処理するか**”という**考え方の部分**が、ロジックです。

【**つながり**】 次に出てくる『CRUD 操作』や『バリデーション』は、このロジックの中で実際に行われる処理です。

9. IDE(アイ・ディー・イー)

ひとことで言うと:プログラムを書く作業を強力に助けてくれる、**高機能なエディタ**。

IDE は Integrated Development Environment(統合開発環境)の略。「コードを書く」「動かす」「間違いを見つける」など、開発に必要な機能が1つのアプリにまとまったものです。

ただのメモ帳が“手書きのノート”だとすると、IDE は **文房具・電卓・辞書・校正ペンが全部ついた多機能デスク**のようなもの。代表例は VS Code(ブイエス コード)です。

- 入力ミスや間違いを、その場で赤く教えてくれる
- 途中まで書くと、続きの候補を出してくれる(入力補助)
- ボタン 1 つでプログラムを実行できる

【つながり】 Claude Code は、こうした開発作業そのものを、あなたと一緒に進めてくれる AI の相棒です。

第3部 — 作るときの「流れ・仕組み」

10. 開発の流れ(全体の手順)

API 設計 → データベース設計 → 認証 → CRUD 操作 → バリデーション → エラーハンドリング → テスト

この流れは「レストランを開店する準備」にそっくりです。大きく「準備する → 作る → 守る・確かめる」の順に進みます。

流れ	やること	レストランの例え
① API 設計	どんな窓口(メニュー)を作るか決める	メニューを決める
② データベース設計	どんな倉庫(保管場所)を作るか決める	倉庫・棚を設計する
③ 認証	正しい利用者か入口で確認する	入口で会員証を確認
④ CRUD 操作	データを作る・読む・変える・消す	料理を作る・出す・変える・下げる
⑤ バリデーション	入力内容がおかしくないか確認する	注文内容を確認する
⑥ エラーハンドリング	トラブルにうまく対応する	売り切れを丁寧に伝える
⑦ テスト	ちゃんと動くか試す	開店前に味見する

◆ 各ステップをもう少し詳しく

- ① **API 設計**: 作り始める前に「どんなお願いを受け付けるか」を表にして決める
- ② **データベース設計**: データをためる「倉庫(DB)」を、Excel の表のような形で設計する
- ③ **認証**: 大切なデータを守るため、「本人ですか?」を入口で確認する門番
- ④ **CRUD 操作**: Create(作る)/Read(読む)/Update(変える)/Delete(消す)の4基本操作。システムの心臓部
- ⑤ **バリデーション**: 「メール欄に“あいいうえお”」のような変な入力を、保存前にはじく

- ⑥ **エラーハンドリング**: 問題が起きても慌てず、「見つかりませんでした」と分かりやすく対応する
- ⑦ **テスト**: 料理の味見と同じ。お客さんに届ける前にミス(バグ)を見つけて直す

【つながり】 ①②が「設計」、③④が「中身を作る」、⑤⑥⑦が「品質を上げる」段階、と覚えると整理できます。

11. エンティティの設計

ひとことで言うと:データベースで扱う「モノ 1 種類」を、表の形で設計すること。

エンティティ(entity)とは「実体・モノ」という意味。システムが扱う“**データのかたまり 1 種類**”のことです。たとえば「会員」「商品」「注文」などが、それぞれ 1 つのエンティティです。

エンティティ設計とは、その 1 種類について「どんな項目を持つか」を決めること。Excel の“表 1 枚”を設計するイメージです。例:「会員」エンティティ

項目(列)	種類	例
id(番号)	数字	1
name(名前)	文字	田中太郎
age(年齢)	数字	28
email(メール)	文字	tanaka@example.com

【つながり】 『データベース設計(②)』を、具体的に「モノ 1 種類ごとの表」に落とし込む作業が、これです。

12. JWT 認証 / JWT トークン

ひとことで言うと:ログイン後に配られる「通行証(トークン)」で本人だと示し続ける仕組み。

JWT は「ジョット」または「ジェイ・ダブリュー・ティー」と読みます(JSON Web Token の略)。

遊園地を想像してください。入口で1回チケットを見せると、**手首にリストバンド(通行証)**を巻いてもらえますよね。中では、いちいちチケットを買い直さなくても、リストバンドを見せるだけで乗り物に乗れます。

この **リストバンドにあたるのが「JWT トークン」**です。ログイン(本人確認)に成功すると、サーバーが“通行証(トークン)”を発行します。以降、利用者はそのトークンを見せるだけで、毎回パスワードを入れ直さなくても「本人ですよ」と示し続けられます。

◆ 流れのイメージ

- ① ID・パスワードでログインする
- ② サーバーが「JWT トークン(通行証)」を発行して渡す
- ③ 以降のお願いには、毎回そのトークンを添える
- ④ サーバーはトークンを確認して「本人だ」と判断する

【つながり】『認証(③)』を実現する代表的な方法の1つです。トークンの中身はJSON ともなっています。

13. グローバル例外ハンドラ

ひとことで言うと:**あちこちで起きるエラー対応を「1 か所」にまとめて受け止める仕組み。**

「例外(exception)」とは、プログラムで起きる“**想定外の問題・エラー**”のこと。「グローバル」は“全体共通の”、「ハンドラ」は“対応係”という意味です。

ビルの各部屋(プログラムのあちこち)で火事(エラー)が起きるたびに、各自バラバラに消火していたら大変です。そこで **建物全体で1つの「総合防災センター」を置いて、どこで起きた問題もそこでまとめて対応する**——それがグローバル例外ハンドラです。

◆ うれしいこと

- エラー対応をあちこちに書かなくてよい(1 か所に集約できる)
- 利用者にはどこで起きても「分かりやすい同じ形」で返せる
- 「謎の英語の画面」ではなく「お探しの会員は見つかりませんでした」と返せる

【つながり】 『エラーハンドリング(⑥)』を、システム全体で“きれいにまとめて”行うための仕組みです。

付録 A — 全体まとめ表

用語	ひとことと言うと	身近な例え
Spring Boot	裏側を素早く作る道具セット	調味料が揃ったキッチン
REST API	やり取りの窓口・お作法	注文を取り次ぐウェ이터
JSON	データの共通の書き方	料理を載せる決まった皿
エンドポイント	窓口一つひとつの住所	注文カウンター
デモ	動かして見せる実演・お試し版	家電量販店の実演
タスク	やるべき作業 1 つ分	To-Do リストの各項目
バッファ	一時的にためる場所・余裕	お風呂の浴槽
ロジック	処理の中身・考え方・手順	料理のレシピ手順
IDE	開発を助ける高機能エディタ	多機能デスク (VS Code)
開発の流れ	設計→作る→守る・確かめる	開店準備の段取り
エンティティ設計	モノ 1 種類を表で設計	Excel の表 1 枚
JWT 認証/トークン	通行証で本人を示す仕組み	遊園地のリストバンド
グローバル例外ハンドラ	エラー対応を 1 か所に集約	総合防災センター

付録 B — 全体のつながり図

これまでの用語は、ひとつの線につながっています。

Spring Boot(便利な道具)を使って、**API 設計**で決めた **REST API**(窓口=**エンドポイント**)を作り、**データベース**(**エンティティ設計**で設計した倉庫)を用意します。

JWT 認証で「本人ですか?」を確かめながら、**ロジック**の中で **CRUD 操作**(作る・読む・変える・消す)を行い、結果を **JSON** で返します。

さらに **バリデーション** で変なデータをはじき、**グローバル例外ハンドラ** でトラブルに備え、最後に **テスト** で確認します。これら全部を **IDE** という道具の上で書いていきます。

これが、Web システム開発の典型的な全体像です。あなたは今、その“地図”を手に入れました。

おわりに — 次の一歩

最初は知らない言葉ばかりで大変だったと思いますが、ひとつずつ見ていけば、ちゃんと 1 枚の絵としてつながっていきます。焦らず、自分のペースで大丈夫です。

◆ こんな進め方がおすすめ

- 分からなくなったら、この文書の「ひとことで言うと」だけ読み返す
- 身近な例え(レストラン/お風呂/遊園地…)とセットで思い出す
- 実際に小さな「会員管理 API」を作ってみると、用語が一気につながる

—— 勉強、いいペースで進んでいます。応援しています! ——